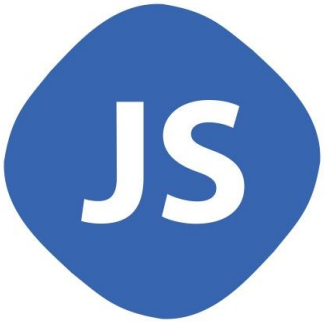


Условные инструкции

- Введение
- Условный оператор if
- Логические операторы в условиях
- Условный оператор if...else
- Условный оператор else if... (несколько условий)
- Оператор switch
- Условный (тернарный) оператор (?:)
- Правило приведения условия к true или false

Введение

Условные операторы - это операторы языка JavaScript, которые в зависимости от некоторого условия позволяют выполнить одно или несколько определённых инструкций.

Формы условных операторов в JavaScript:

- условный оператор **if** (с одной ветвью);
- условный оператор **if...else** (с двумя ветвями);
- условный оператор **else if...** (с несколькими ветвями);
- тернарный оператор (**?:**);
- оператор выбора **switch**.

Условный оператор if

Условный оператор выполняет тот или иной **блок кода** в зависимости от того, верно ли условие.

> ***К сведению***. В данном случае сочетание **блок кода** это не просто игра слов, но **управляющая инструкция** JavaScript (поговорим в следующем



уроке).

Синтаксис без фигурных скобок

```
if (condition)
      statement
```

Параметры

- **condition** - выражение, являющееся условием для проверки. Должно равняться **true** или **false** (или быть приведено к одному из этих значений);
- **statement** - инструкция, которая выполняется, если условие является **true** или приведено к нему.

Если необходимо выполнить несколько инструкций, то их необходимо **поместить в фигурные скобки**.

Синтаксис с фигурными скобками

```
if (condition) {
      statement 1
      ...
      statement n
}
```

Рекомендуется, **использовать фигурные скобки даже когда используется одна инструкция**.

example_1. Простой оператор if

```
<script>
    // шаблон регулярного выражения RegExp
    var reg = /^[а-яё-]{3,25}$/i;
    // инициализируем переменную
    // var surname = "Бендер-Задунайский";
    var surname = prompt("Введите фамилию. Символы [а-яё-] в
```

```
количестве от 3 до 25");
    // проверка соответствия вернет true или false
    var res = reg.test(surname);
    // если res == true
    if (res) {
        alert ("Вы успешно справились с заданием!");
    }
</script>
```

Логические операторы в условиях

В JavaScript есть **логические операторы**:

- **||** (ИЛИ)
- **&&** (И)
- **!** (НЕ)

Несмотря на своё название, данные операторы могут применяться к **значениям любых типов**. Полученные результаты также могут иметь различный тип.

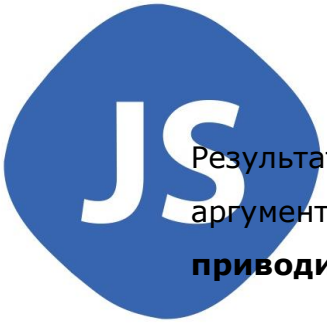
|| (ИЛИ)

Оператор **ИЛИ** выглядит как двойной символ вертикальной черты **||**:

```
result = a || b;
```

Традиционно в программировании существует четыре возможные логические комбинации:

```
alert( true || true );  // true
alert( false || true );  // true
alert( true || false );  // true
alert( false || false ); // false
```



Результат операций всегда равен **true**, за исключением случая, когда оба аргумента **false**. Если значение не логического типа, то оно к нему **приводится в целях вычислений**.

Например, число 1 будет воспринято как **true**, а 0 – как **false**:

(1 || 0) будет приведено к виду (true || false)

Обычно оператор **||** (или несколько операторов **||**) используется в **if** для проверки истинности **любого из заданных условий**.

```
let num = 12;  let isChecked = true;
if (num < 10 || num > 18 || isChecked) {/* ... */}
```

> ***Обратите внимание***. **ИЛИ ||** находит **первое истинное значение**.

При выполнении **ИЛИ ||** с несколькими значениями:

```
result = value1 || value2 || value3;
```

оператор || выполняет следующие действия:

- Вычисляет операнды слева направо.
- Каждый операнд конвертирует в логическое значение. Если результат true, останавливается и возвращает исходное значение этого операнда.
- Если все операнды являются ложными (**false**), возвращает последний из них. Значение возвращается в исходном виде, без преобразования.

Другими словами, цепочка **ИЛИ ||** возвращает **первое истинное значение или последнее**, если такое значение не найдено.

example_2. Поведение логического ИЛИ

```
<script>
      console.log( 1 || 0 ); // 1
      console.log( true || 'also true' ); // true
      console.log( null || 1 ); // 1 (первое истинное значение)
```



```
    console.log( null || 0 || 'hello' ); // 1 (первое истинное
    значение)
    console.log( undefined || null || 0 ); // 0 (поскольку все ложно,
    возвращается последнее значение)
</script>
```

Сокращённое вычисление

Операндами могут быть как **отдельные значения**, так и **произвольные выражения**. **ИЛИ ||** вычисляет их слева направо. Вычисление останавливается при достижении **первого истинного значения**.

> ***К сведению***. Этот процесс называется **сокращённым вычислением**, поскольку **второй операнд** вычисляется только в том случае, если первого **недостаточно** для вычисления всего выражения.

Это хорошо заметно, когда выражение, указанное в качестве второго аргумента, имеет **побочный эффект**, например, изменение переменной.

В приведённом ниже примере переменная **x** не изменяется:

```
var x;
true || (x = 1);
// (x = 1) не станет вычисляться
alert(x); // undefined
```

Если бы первый аргумент имел значение **false**, то **||** приступил бы к вычислению второго **и выполнил операцию присваивания**:

```
var x;
false || (x = 1);
// в этом примере (x = 1) вычисляется
alert(x); // 1
```

На месте первого аргумента может быть сложное выражение или, например, функция, тогда поведение программы становится менее очевидным.

```
// первым аргументам используем функцию
var x;

myUserFun() || (x = 1);

// в этом примере (x = 1) вычисляется

alert(x); // ?
```

Присваивание – лишь один пример. Но, могут быть и другие побочные эффекты, которые не проявятся, если вычисление до них не дойдёт.

Такой вариант использования **||** является **аналогом if**. Первый операнд преобразуется в **логический**, если он оказывается ложным, начинается вычисление второго.

В большинстве случаев лучше использовать классический **if**, чтобы облегчить понимание кода, но иногда это может быть удобно.

&& (И)

Оператор **И** пишется как два амперсанда **&&**:

```
result = a && b;
```

В традиционном программировании **И** возвращает **true**, если оба аргумента истинны, а иначе – **false**:

```
alert( true && true );  // true

alert( false && true );  // false

alert( true && false );  // false

alert( false && false );  // false
```

Пример с **if**:

```
let hour = 12; let minute = 30;

if (hour == 12 && minute == 30) {/*...*/}
```

Обратите внимание. **И &&** находит первое ложное значение.

При нескольких подряд операторах **И**:

```
result = value1 && value2 && value3;
```

оператор && выполняет следующие действия:

- Вычисляет операнды слева направо.
- Каждый операнд преобразует в логическое значение. Если результат **false**, останавливается и возвращает исходное значение этого операнда.
- Если все операнды были истинными, возвращается последний.

Другими словами, **И** возвращает **первое ложное значение**, или **последнее**, если ничего не найдено.

Вышеуказанные правила схожи с поведением **ИЛИ**. Разница в том, что **И** возвращает **первое ложное** значение, а **ИЛИ** – **первое истинное**.

example_3. Поведение логического И

```
<script>
    // если первый операнд истинный, будет возвращен второй
    console.log( 1 && 0 ); // 0
    console.log( 1 && 5 ); // 5
    // если первый операнд ложный, он будет возвращен, второй операнд игнорируется
    console.log( null && 5 ); // null
    console.log( 0 && "Vivat the king" ); // 0
</script>
```

Можно передать несколько значений подряд. В таком случае возвратится **первое ложное значение**, на котором остановились вычисления.

```
alert( 1 && 2 && null && 3 ); // null
```

Когда все значения **верны**, возвращается **последнее**.

```
alert( 1 && 2 && 3 ); // 3
```

> ***Важно***. Приоритет оператора **И &&** больше, чем **ИЛИ ||**, так что он выполняется раньше.

Как и оператор **ИЛИ ||**, **И &&** иногда может заменять **if**.

```
let x = 1;

(x > 0) && alert('Hello, world!');
```

Действие в правой части **&&** выполнится только в том случае, если до него дойдут вычисления. То есть, **alert**() сработает, если в левой части (x > 0) будет **true**. Получился аналог:

```
let x = 1;

if (x > 0) {

    alert('Hello, world!');

}
```

Однако, как правило, вариант с **if** лучше **читается** и **воспринимается**. Он более очевиден, поэтому лучше использовать его.

! (НЕ)

Оператор **НЕ** представлен восклицательным знаком **!**.

Синтаксис довольно прост:

```
result = !value;
```

Оператор принимает один аргумент и выполняет следующие действия:

- Сначала **приводит аргумент к логическому типу true/false**.
- Затем **возвращает** противоположное значение.

Например:

```
alert( !true ); // false

alert( !0 ); // true
```

```
alert( !'Балалайка' ); // false
```

В частности, двойное **НЕ !!** используют для преобразования значений к логическому типу:

```
alert( !!"It's My Life" ); // true
alert( !!null ); // false
```

То есть первое **НЕ** преобразует значение в логическое значение и возвращает обратное, а второе **НЕ** снова инвертирует его. В конце мы имеем простое преобразование значения в логическое.

Есть немного более подробный способ сделать то же самое – встроенная функция **Boolean**:

```
alert( Boolean("my-string") ); // true
alert( Boolean(null) ); // false
```

> ***Важно***. Приоритет **НЕ !** является **наивысшим из всех логических операторов**, поэтому он всегда выполняется **первым**, перед **&&** или **||**.

example_4. Отрицание в условиях

```
<script>
    // определим переменные для хранения данных
    let name = "Джон Джозеф Траволта"; // не вписывается в длину
    let login = "john-travolta";
    let pwd = "holly@wood!"; // символ ! - не валиден
    // определим шаблоны длч проверки данных
    let namePattern = /^[a-яё ]{3,15}$/i;
    let loginPattern = /^[a-z0-9-]{3,15}$/i;
    let pwdPattern = /^[a-z_@#$0-9-]{5,15}$/i;
    // выполним проверку данных на валидность
```

```
    let nameValid = namePattern.test(name);
    let loginValid = loginPattern.test(login);
    let pwdValid = pwdPattern.test(pwd);
    // выведем сообщение, если валидация не пройдена
    if (!nameValid) console.log("Значение Имя не валидно");
    if (!loginValid) console.log("Значение Логин не валидно");
    if (!pwdValid) console.log("Значение Пароль не валидно");
</script>
```

Условный оператор if...else

Оператор **if...else** используется, когда необходимо при истинности условия выполнить одни инструкции, а при ложности - другие.

Синтаксис

```
if (condition){
    statements
}else {
    statements
}
```

Например, выведем в консоль сообщение о том, является ли число четным или нет:

```
if (number % 2) {
    console.log('Число нечётное!');
} else {
    console.log('Число чётное!');
}
```



Так как параметр **condition** является выражением, то в круглые скобки оператора **if** можно передавать функции, например: **confirm**() и методы, например: **RegExp.test**(string). Проверяем.

example_5. Оператор if - else

```
<script>
	// шаблон RegExp
	var reg = /^[a-яё-]{3,25}$/i;
	// инициализируем переменную
	var surname = prompt("Введите фамилию. Символы [a-яё-] в количестве от 3 до 25");
	// проверка соответствия вернет true или false
	if (reg.test(surname)) {
		// если true
		alert ("Вы успешно справились с заданием!");
	} else {
		// если false
		alert ("Бумс... Что-то пошло не так?");
	}
</script>
```

Условный оператор else if... (несколько условий)

Синтаксис

```
if (condition1){
	statements
} else if (condition2){
	statements
```

```
} else if (conditionN) {
    statements
} else {
    statements
}
```

example_6. Оператор if - else if

```
<script>
    // дадим пользователю право определить порядок выполнения кода
    if (confirm("Вы хотите изучать JavaScript?")) {
        alert ("Очень хорошо!\nПриготовьтесь погрузиться в
        удивительный мир frontend разработки!!!") ;
    } else if (confirm("Может тогда PHP?")){
        alert ("Прекрасно ;)");
    } else if (confirm("Ну может из Пастернака что-нибудь??")) {
        document.write (poem);
    } else {
        alert("Вам трудно угодить ...")
    }
</script>
```

example_7. Инструкций if - else if может быть сколько угодно

```
<script>
     // определим номер дня недели
     const dayNumber = new Date().getDay();
     // console.dog(dayNumber);
     // преобразуем номер в название
     if (dayNumber === 0) {
          day = "Воскресенье";
```

```
    } else if (dayNumber === 1) {
        day = "Понедельник";
    } else if (dayNumber === 2) {
        day = "Вторник";
    } else if (dayNumber === 3) {
        day = "Среда";
    } else if (dayNumber === 4) {
        day = "Четверг";
    } else if (dayNumber === 5) {
        day = "Пятница";
    } else {
        day = "Суббота";
    }
    alert ("Сегодня " + day);
</script>
```

Оператор switch

Оператор **switch** предназначен для выполнения **одного варианта инструкций из нескольких** в зависимости от значения выражения. Выбор того или иного варианта определяется посредством строгого равенства результата выражения значению случая (**case**).

Синтаксис

```
switch (expression) {
    case label1:
        statements1
        [break]
    case label2:
```

```
        statements2
        [break]
    ...
    case labelN:
        statementsN
        [break]
    default:
        statements_def
        [break]
}
```

Параметры

- **expression** - выражение для сравнения.
- **labelN** - значение, с которым сравнивать.
- **statementsN** - ветка кода, на которую перейдет управление при выполнении условия **expression** = **labelN**.
- **statements_def** - ветка кода, которая будет выполнена, если **expression** не совпадет ни с одним значением **labelN**.

example_8. Оператор switch

```
<script>
    let day;
    // тестируем оператор switch
    switch (new Date().getDay()) {
        case 0:
            day = 'Воскресенье';
            break;
        case 1:
            day = 'Понедельник';
            break;
```

```
            case 2:
                  day = 'Вторник';
                  break;
            case 3:
                  day = 'Среда';
                  break;
            case 4:
                  day = 'Четверг';
                  break;
            case 5:
                  day = 'Пятница';
                  break;
            default:
                  day = 'Суббота';
      }
      alert ("Сегодня " + day);
</script>
```

Конструкция **switch** служит для сравнения значения на равенство с различными вариантами. При этом равенство подразумевается в смысле оператора "===", **сравнивать с регулярным выражением** или как-то еще **switch** не умеет.

Если совпадение найдено, то соответствующий код исполняется до оператора **break**, который прекращает выполнение **switch** и передает управление дальше.

example_9. В разделе case может быть не только число

```
<script>
     let msg = `
Напишите, кто вы по темпераменту: \n
```

```
- Флегматик
- Сангвиник
- Холерик
- Флегматик
`;
    switch (prompt(msg)) {
        case "Флегматик":
            temp = '<h2>Флегматик</h2>В своей деятельности они всегда склонны к порядку и любят привычную для них обстановку...';
            break;
        case "Сангвиник":
            temp = '<h2>Сангвиник</h2>Это крайне непоседливые, энергичные, легкие на подъем люди, которые очень любят движение...';
            break;
        case "Холерик":
        temp = '<h2>Холерик</h2>Холерики, по сравнению с сангвиниками, вспыльчивы, непостоянны, агрессивны и импульсивны...';
        break;
        case "Меланхолик":
            temp = '<h2>Меланхолик</h2>Достаточно ранимые, обидчивые и скрытые личности. Они склонны к глубоким переживаниям и грустным мыслям.';
            break;
        default:
            temp = '<h2>Вы неврастеник! ;)</h2>';
    }
    document.querySelector("div#app").innerHTML = temp;
</script>
```



Ключевое слово **break** не является обязательным, но если вы не будете использовать его в данном примере, то в качестве значения переменной **temp** всегда будете получать **дефолтное значение**.

Рассмотрим пример работы **switch** без **break**. В такой ситуации будут последовательно выполнены последние три вызова метода **alert**().

example_10. Оператор switch без break

```
<script>
    let x = 2 + 2;
    switch (x) {
        case 3:
        alert('Маловато будет!');
    case 4:
        alert('Гадал?');
    case 5:
        alert('Перебор...');
    default:
        alert('Кто здесь?');
    }
</script>
```

Несколько значений **case** можно группировать.

```
switch (2 + 2) {
    case 4:
        alert('Верно!');
        break;
    case 3:
    case 5:
        alert('Неверно!');
```

```
        break;
    default:
        alert('Я таких значений не знаю');
}
```

Условный (тернарный) оператор (?:)

Тернарный оператор – оператор JavaScript, который можно использовать, когда необходимо в зависимости от условия выполнить **одно из двух** заданных выражений.

Синтаксис

```
condition ? statement 1 : statement 2;
```

Тернарный оператор состоит из **трех операндов**, которые разделяются с помощью символов **?** и **:**. Условие тернарного оператора задаётся в первом операнде. Его также можно заключить в скобки **()**.

Если условие равно **true**, или может быть приведено к этому значению, будет выполнено выражение **statement 1**, иначе - выражение **statement 2**.

```
(age > 14) ? console.log('Добро пожаловать') : console.log('Доступ к контенту закрыт из-за возрастных ограничений');
```

или так:

```
msg = (age > 14) ? 'Добро пожаловать' : 'Доступ к контенту закрыт из-за возрастных ограничений';

console.log(msg);
```

example_11. Тернарный оператор

```
<script>

    // объявим данные для проверки

    let name = "Denis1";
```

```
    let login = "master";
    let pwd = "Denis123@qw^";
    // определим шаблоны проверки
    let name_pattern = /^[a-za-яё]{3,10}$/ig;
    let login_pattern = /^[a-zA-Z0-9 ]{5,10}$/g;
    let pwd_pattern = /^[a-zA-Z0-9@#$%^&*]{5,15}$/g;
    // предположим, что проверяться будет не один пользователь
    // пригодится в следующих уроках
    let user = [];
    // первый пользователь
    user[0] = {};
    // выполняем валидацию данных
    user[0].name = name.match(name_pattern) ? name : "Поле name не
    валидно";
    user[0].login = login.match(login_pattern) ? login : "Поле login
    не валидно";
    user[0].pwd = pwd.match(pwd_pattern)? pwd : "Поле pwd не валидно";
    // вывод массива в консоль
    console.dir(user);
</script>
```

Правило приведения условия к true или false

Если выражение в условии оператора **if** не равно **true** или **false**, то **JavaScript приведёт его к одному из этих значений**. Данное действие он выполняет с помощью, так называемого **правила лжи**.

Смысл данного правила: **любое выражение является true**, кроме следующих значений:

- **false** (ложь);
- **""** или **''** (пустая строка);
- **NaN** (специальный числовой тип данных "не число");
- **0** (число "ноль");
- **null** ("пустое" значение);
- **undefined** ("неопределённое" значение).

Например, выведем в консоль браузера приветственное сообщение, в зависимости от того, какое значение хранится в переменной **user**:

```
if (user) {
    console.log('Привет, ' + user + '!');
} else {
    console.log('Привет, гость!');
}
```

Если, например, переменная **user** будет содержать пустую строку, то по правилу лжи, она будет приведена к значению **false**. Следовательно, в консоль будет выведено сообщение "Привет, гость!".

А если переменная **user** будет содержать строку "Denis", то выражение в условии будет приведено к значению **true**. В результате, в консоли отобразится сообщение "Привет, Denis!".

example_12. Приведение условий к логическому значению

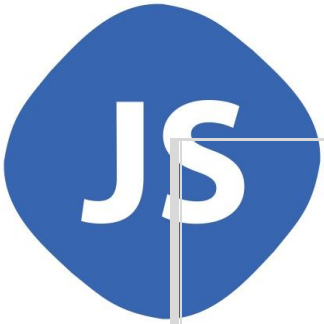
```
<script>
    // 1 вариант, user - не объявлена
    /*
    2 вариант, user объявлена, но инициализирована пустой строкой
    var user = ""; // false
    */
```

```
    /*
    3 вариант, user объявлена, и инициализирована строкой
    var user = "Denis"; // true
    */
    if (user) {
        console.log('Привет, ' + user + '!');
    } else {
        console.log('Привет, гость!');
    }
</script>
```

Перед подготовкой к выполнению самостоятельных заданий выполним еще один пример.

example_13. Обработка заказа

```
<script>
    // на сайте действует скидка - 20%
    // если сумма заказа свыше 10.000, то скидка - 25%
    let orderJSON = `
    {
        "user":{
        "name":"Евгений",
        "login":"master",
        "email":"master@gmail.ru"},
        "info":{
            "date":"06.01.23",
            "price":12500,
            "newprice":"undefined"
        },
```



```
        "course":[
            {
                "id":41,
                "name":"Frontend- и backend- программирование: PHP и
                JavaScript",
                "hours":90,
                "certificate":"yes",
                "price":7500
            },
            {
                "id":12,
                "name": "Веб анимация средствами JavaScript и CSS",
                "hours":30,
                "certificate":"yes",
                "price":5000
            }
        ]
    }`;
    // парсим строку JSON в JS объект
    let order = JSON.parse(orderJSON);
    // текущая цена без учета скидок
    price = order.course[0].price + order.course[1].price;
    // вычисляем действующий процент скидки
    if (price > 10000) {
        discount = 0.25;
    } else {
        discount = 0.2;
    }
```

```
    // вычисляем новую цену
    newprice = price - (price * discount);
    // записыаеи в массив новую цену
    order.info.newprice = newprice;
    console.dir(order);
</script>
```

Вообще говоря, обычной практикой считается назначение **статуса** для зарегистрированного пользователя. И уже, исходя, из существующего статуса, осуществлять автоматическое регулирование **системы скидок**. Займемся в одной из самостоятельных работ.

Самостоятельные работы к уроку:

- Нахождение корней квадратного уравнения.
- Валидация форм с помощью регулярных выражений.
- Обработка данных заказа.
- Улучшаем реализацию игры Кубики.